

Complete Frontend Flow — Beginner's Guide

See also:

- [SHARED TYPES.md](#) — `Route` , `RouteNode` , `FactNode` interfaces and spritesheet coordinate system
- [ACCESSIBILITY.md](#) — accessibility `localStorage` schema and CSS implementation

What is the frontend?

The frontend is a **Next.js** (React) web app. It is what the user sees and interacts with on their phone or tablet. It talks to the backend over HTTP using `fetch` , stores small pieces of data in `localStorage` and cookies, and renders different pages based on the URL.

Architecture Overview

```
Next.js App (frontend/src/app/)
├── layout.tsx          - Root HTML shell, wraps every page
├── middleware.ts       - URL allowlist / redirect guard (runs before every page)
├── page.tsx            - "/" Session ID entry page
├── home/page.tsx       - "/home" Home menu
├── route/page.tsx      - "/route" Active walking route game screen
├── pollinator-collection/page.tsx - "/pollinator-collection" Collected pollinators
view
├── accessibility/page.tsx - "/accessibility" Accessibility settings
├── redirecting/page.tsx  - "/redirecting" Countdown then redirect to "/"
├── components.tsx       - All shared UI components
└── services/
    ├── jwtService.ts    - JWT storage + all authenticated API calls
    └── routeService.ts  - Completed-routes storage + route selection helpers
```

Shared Types

[shared/types.ts](#)

Defines the data shapes shared between the frontend and backend:

```
interface RouteNode {
  id: string;    // the node's stable key (e.g. "1.0") – used as image coords
  text: string;  // the display label for this stop
}

interface FactNode {
  id: string;    // nodeId from DB
  text: string;  // fact text
  pollinator: string;
}
```

```
interface Route {
  routeId: string;
  routeNodes: RouteNode[];
  factNodes: FactNode[];
}
```

`shared/data/Routes.json` — DELETED / UNUSED

This file previously mapped pollinator names to spritesheet coordinate arrays and was used by `home/page.tsx` and `pollinator-collection/page.tsx`. Both pages now fetch this data from `/api/get-pollinator-names` instead. The file is no longer imported anywhere and has been removed.

Layer 1 — Root Layout

[frontend/src/app/layout.tsx](#)

Wraps every page in the app. Sets the page `<title>` to "Pollinator Habitat", links the favicon, and renders a minimal `<html><body>{children}</body></html>` shell. No navigation bar or global state — just the HTML skeleton.

Layer 2 — URL Middleware (Route Guard)

[frontend/src/middleware.ts](#)

Runs **before every page request** in Next.js (server-side, before the page renders).

Allowed paths (allowlist):

```
/
/home
/redirecting
/accessibility
/pollinator-collection
/route
```

Logic:

1. Gets the `pathname` from the incoming request URL
2. If the path is in `PUBLIC_PATHS` → allows through with `NextResponse.next()`
3. Also allows through: paths starting with `/_next` (Next.js internals), `/api`, or paths matching a file extension regex (`/\.[^\/]+$/` — e.g. images, CSS)
4. Any other URL → redirects to `/redirecting` (the countdown page)

This prevents users from manually typing arbitrary URLs as a security measure.

Layer 3 — Pages

[frontend/src/app/page.tsx](#) — "/" Session Entry

Purpose: The first page a user sees. They type in a session code and validate it to get a JWT token.

State:

Variable	Type	Purpose
sessionCode	string	The value typed in the input field
error	string	Error message to display below the form
loading	boolean	Disables the button and shows "Validating..." while the API call is in flight
isValidated	boolean	Once true, hides the form and shows a HomeButton

Input field behavior:

- Uses the `TextInput` component (not a raw `<input>`)
- `inputMode="numeric"` — shows number keyboard on mobile
- `minLength={9} / maxLength={9}` — session codes are now **9 digits**
- `onChange` strips all non-digit characters and slices to 9 characters; also clears `error` state if one is currently set
- Disabled while `loading === true`

Label: Shows "Please Enter A Session Code" before validation; shows "Successfully Connected" after validation.

`handleValidateSession()` :

1. If `sessionCode.trim()` is empty → sets error "Please enter a session code", returns early
2. If `sessionCode.length !== 9` → sets error "Error: Session code must be 9 digits", returns early
3. Sets `loading: true`, clears `error`
4. POST to ``${API_BASE}/api/`` with body `{ sessionId: sessionCode }` and `Content-Type: application/json`
5. Reads the response as raw text first (`response.text()`)
6. If `response.ok` :
 - Tries to `JSON.parse(text)` — if that fails, sets error "Server error: Invalid response format"
 - On success: stores `data.token` in `localStorage` as "authToken", stores `sessionCode` in `localStorage` as "sessionId", sets `isValidated: true`
7. If not `response.ok` :
 - Tries to parse error JSON — if the parse itself throws, sets error "Server error: Invalid session code" and returns; otherwise shows `errorData.error` or falls back to "Invalid session code"
8. On network failure → sets error "Failed to validate session. Please try again."
9. finally : sets `loading: false`

Note: This page stores the token under the key "authToken". The `jwtService.ts` file stores it under the key "pollinator_jwt_v1". These are **two separate localStorage keys** — `page.tsx` manages its own token independently of `jwtService.ts`.

Submit button: `EventButton` with text 'Submit' (was previously `InteractiveButton` with text 'Validate').

After validation: Shows a `HomeButton` (links to `/home`).

[frontend/src/app/home/page.tsx](#) — `"/home"` Home Menu

Purpose: The main menu. Shows navigation buttons and a pollinator progress counter.

State:

Variable	Line	Type	Purpose
<code>totalPollinatorCount</code>	L11	number	Total number of pollinators in the game — fetched from API

On render (synchronous):

1. [L10](#) — Calls `getCompletedRoutes()` from `routeService.ts` — reads the completed routes from `localStorage` (with cookie fallback)
2. [L13](#) — Calls `setReplayRoute(null)` — clears any replay state when returning to home
3. [L12](#) — Displays: `"Pollinators Collected: <completedRoutes.length>/<totalPollinatorCount>"`

[L15–19](#) `useEffect` on mount:

- Calls `apiFetchAuthed<{ pollinator: { name: string, coord: string }[] }>("/api/get-pollinator-names", { method: "POST" })`
- On success: sets `totalPollinatorCount` to `data.pollinator.length`
- On error: silently swallows (count stays `0`)

Buttons rendered:

- "Start New Pollinator Path" → links to `/route`
- "Accessibility" → links to `/accessibility`
- "Pollinator Collection" → links to `/pollinator-collection`

[frontend/src/app/route/page.tsx](#) — `"/route"` Route Game Screen

Purpose: The main game screen. Fetches a route from the backend, walks the user through each stop, then through fact cards, then lets them complete the route and get a new one.

State ([L14–20](#)):

Variable	Line	Type	Purpose
<code>imageCoords</code>	L14	string	Coordinate string (e.g. "1.0") for the sprite-sheet image
<code>pollText</code>	L15	string	The text shown in the main <code>RouteText</code> display
<code>route</code>	L16	<code>Route[]</code>	Array holding the current route (always 0 or 1 item)
<code>routeIndex</code>	L17	number	Current position — 0 through <code>routeNodes.length + factNodes.length - 1</code>
<code>collectorPopUpGuard</code>	L18	boolean	Prevents the <code>CollectorPopUp</code> from showing again after dismissed
<code>surveyPopUpGuard</code>	L19	boolean	Prevents the <code>PartySizeSurveyPopUp</code> from showing again after dismissed

ttsEnabled	L20	boolean	Whether TTS is currently on — read from localStorage on mount, passed as prop to all four layout functions
------------	---------------------	---------	--

[L53-94](#) Initial load `useEffect` (empty dep array):

1. [L63](#) — Calls `getReplayRoute()` from `routeService` — if non-null, uses that route directly instead of calling `/api/start-game`
2. [L56-61](#) — Reads `"authToken"` from `localStorage` — the JWT stored by `page.tsx` at session validation. If missing → sets `pollText: "No session token. Please validate your session."` and returns; otherwise calls `saveJwt(token)` to copy it into `"pollinator_jwt_v1"` for `apiFetchAuthenticated`
3. [L70-74](#) — Calls `apiFetchAuthenticated<{ player: { activeRoute: Route | null } }>` (`"/api/start-game", { method: "POST" }`) (skipped if replay route exists)
4. [L82-86](#) — If no route → sets `pollText: "No route available."`; otherwise sets `route`, `pollText`, and `imageCoords` from the first `routeNode`

TTS helpers — see the [Text-to-Speech section](#) below for full detail:

- [L27-33](#) `isTTSEnabled()` — reads `localStorage["accessibilitySettings"].TTS`; returns `false` on parse failure
- [L35-42](#) `getTextForIndex(index)` — resolves which text string corresponds to a given `routeIndex`
- [L44-51](#) `speakIfEnabled(text)` — if TTS is on, cancels any current utterance then speaks at `rate = 0.75`
- [L22-25](#) TTS init `useEffect` — sets `ttsEnabled` on mount AND returns `() => window.speechSynthesis.cancel()` as cleanup

[L97-114](#) `handleButtonClick(i)` :

- [L98-103](#) `i === 0` (Previous): only acts if `routeIndex > 0` — `newIndex = routeIndex - 1` → `speakIfEnabled(getTextForIndex(newIndex))` → `setRouteIndex(newIndex)`
- [L104-109](#) `i === 1` (Next): only acts if `routeIndex < routeNodes.length - 1 + factNodes.length - newIndex = routeIndex + 1` → `speakIfEnabled(getTextForIndex(newIndex))` → `setRouteIndex(newIndex)`
- [L110-113](#) `i === 2` (Home): calls `window.speechSynthesis.cancel()` then `handleHomeClicked()`

[L129-169](#) `handleCompleteRoute()` :

1. Guards: if `route` is empty → logs error and returns
2. [L135](#) Calls `addCompletedRoute(route[0])` — passes the full `Route` object
3. [L138-139](#) Sets `pollText: "Loading new route..."`, resets `routeIndex: 0`
4. [L142-149](#) Calls `apiFetchAuthenticated` on `/api/complete-route`
5. If `result.newActiveRoute` is null → sets `pollText: "No route available."`
6. [L156-164](#) Otherwise: sets new route, resets index, resets `collectorPopUpGuard: false`, calls `setReplayRoute(null)`, sets new `pollText` and `imageCoords`

[L116-127](#) `handleHomeClicked()` :

- If user is at fact display phase (`routeIndex - routeNodes.length >= -1`), calls `addCompletedRoute(route[0])` before navigating home

*Note: `surveyPopUpGuard` is **not** reset here — the survey only shows once per app load.*

L171–184 Route index `useEffect` : Runs whenever `route` or `routeIndex` changes:

- If `routeIndex < totalRoute` → sets `pollText` and `imageCoords` from `routeNodes[routeIndex]`
- Else (fact phase) → sets `pollText` from `factNodes[factIndex]` (no image update)

L186–193 Derived display flags (computed fresh each render):

Flag	Line	Condition	Meaning
<code>isRouteEnd</code>	L186	<code>routeIndex >= routeNodes.length + factNodes.length - 1</code>	At the last fact card
<code>isRouteBeginning</code>	L190	<code>routeIndex === 0</code>	At the first stop
<code>isFactDisplay</code>	L191	<code>routeIndex - routeNodes.length >= 0</code>	Past all route stops, in fact phase
<code>isCollectionTime</code>	L192	<code>routeIndex === routeNodes.length - 1</code>	At the very last route stop (before facts)
<code>isSurveyTime</code>	L193	<code>isFactDisplay && isRouteEnd && !surveyPopUpGuard</code>	At the final fact card
<code>showCollectorPopUp</code>	L218	<code>isCollectionTime && !collectorPopUpGuard</code>	Derived flag for rendering
<code>showSurveyPopUp</code>	L219	<code>isSurveyTime && !surveyPopUpGuard</code>	Derived flag for rendering

L207–216 Which HTML layout to render:

Condition	Layout function	Line
<code>isRouteBeginning</code>	RouteBeginningHTML — logo, Home + Settings, stop text, TTSButton (if on), image, Next only	L209
<code>isRouteEnd</code>	RouteEndHTML — logo, Home + Settings, factHeader, fact text, TTSButton (if on), "Start New Route", Previous only	L211
<code>isFactDisplay</code> (not end)	RouteFactHTML — logo, Home + Settings, factHeader, fact text, TTSButton (if on), Previous + Next	L213
Default (mid-route)	RouteNormalHTML — logo, Home + Settings, breadcrumb trail, stop text, TTSButton (if on), image, Previous + Next	L215

L226–244 Pop-ups:

- `CollectorPopUp` — shown when `showCollectorPopUp` . `isNew` prop is `getReplayRoute() === null` . `dataTestId="CollectorPopUp"` . Dismissed by "Continue" → `setCollectorPopUpGuard(true)` .
- `PartySizeSurveyPopUp` — shown when `showSurveyPopUp` . `dataTestId="PartySizeSurveyPopUp"` . `onContinue` is `async` — `await sendSurveyResponseToDatabase(...)` is awaited before `setSurveyPopUpGuard(true)` .

L195–204 Breadcrumb (previousRouteNodeNames): Built in the component body — concatenates all prior stop names with ► separator. Only computed when !isFactDisplay .

L221 isNewPollinator : getReplayRoute() == null — passed as isNew prop to CollectorPopUp .

L201–204 Fact header: "Did you know?" . Passed as a prop to RouteFactHTML and RouteEndHTML .

[frontend/src/app/pollinator-collection/page.tsx](#) — `"/pollinator-collection"`

Purpose: Shows which pollinators the user has discovered and which remain hidden.

State:

Variable	Line	Type	Purpose
completedRoutes	L18	Route[]	Full Route objects for discovered pollinators
incompleteRoutes	L19	string[]	Pollinator names not yet discovered
allPollinators	L20	{ name: string, coord: string }[]	All pollinators from the API (used to derive incomplete list and silhouette images)

L9–15 getIncompleteRoutes(completedRoutes, allPollinators) : Derives which pollinators haven't been discovered. Gets all names from allPollinators , filters out those whose name matches the last routeNode.text of any completed route.

L22–26 useEffect — fetch all pollinators (mount only):

- Calls apiFetchAuthed<{ pollinator: { name: string, coord: string }[] }>("/api/get-pollinator-names", { method: "POST" })
- On success: sets allPollinators state
- On error: silently swallows

L28–47 useEffect — load completed routes (mount only):

1. Calls getCompletedRoutes() to read Route[] from localStorage/cookie; sets completedRoutes
2. Registers two event listeners for live updates:
 - "storage" event — fires when another browser tab changes localStorage → re-reads routes
 - "completedRoutesUpdated" custom event — fires from same-tab addCompletedRoute() → uses event.detail (full routes array)
3. Returns cleanup function removing both listeners

L49–51 useEffect — derive incomplete routes:

- Depends on [completedRoutes, allPollinators]
- Calls setIncompleteRoutes(getIncompleteRoutes(completedRoutes, allPollinators))

L53–55 onReplaySet(route) : Calls setReplayRoute(route) — stores the replay route before navigating to /route .

Rendering:

- "Discovered Pollinators" section: renders each completed route using the last routeNode's `id` for spritesheet coords and `text` for the name. Each has a `ReplayButton` that calls `onReplaySet(route)`.
- "Unknown Pollinators" section: renders each incomplete pollinator name using `allPollinators` to look up its `coord`. Shows `PollinatorImage` with `isHidden: true` (solid black silhouette) and `altText: "Unknown Pollinator"`.

[frontend/src/app/accessibility/page.tsx](#) — `"/accessibility"`

Purpose: Lets the user toggle accessibility settings that persist across sessions.

State:

Variable	Purpose
<code>highContrast</code>	Toggles high-color-contrast CSS class on <code><html></code> element
<code>TTS</code>	Toggles TTS-option-switch CSS class on <code><html></code> element; consumed by <code>route/page.tsx</code> to enable inline TTS buttons
<code>buttonPressReadAloud</code>	Stored in <code>localStorage</code> and state — no UI control and no active behaviour
<code>autoReadAloud</code>	Stored in <code>localStorage</code> and state — no UI control and no active behaviour
<code>accessibilitySettingsIsLoaded</code>	Guard to prevent saving to <code>localStorage</code> before initial load completes

Note: Only `highContrast` and `TTS` have rendered checkbox controls. `buttonPressReadAloud` and `autoReadAloud` are kept in state and persisted to `localStorage` for future use but have no UI toggles. `TTS` is consumed by `route/page.tsx` — when enabled, each navigation step renders a `TTSButton` that speaks the current node text.

Three `useEffect` s:

1. [L22-32](#) **On mount** — reads `"accessibilitySettings"` from `localStorage`, parses JSON, sets all four state values. Sets `accessibilitySettingsIsLoaded: true`
2. [L34-43](#) **On settings change** — depends on all four settings + `accessibilitySettingsIsLoaded`. Skips if not loaded yet. Writes `{ highContrast, TTS, buttonPressReadAloud, autoReadAloud }` to `localStorage` as `"accessibilitySettings"`
3. [L45-62](#) **On DOM sync** — depends on `[highContrast, TTS, buttonPressReadAloud, autoReadAloud]`. [L49-53](#) Adds/removes `'high-color-contrast'` on `<html>`. [L54-58](#) Adds/removes `'TTS-option-switch'` on `<html>`. Only these two have DOM effects; the other two are in the dep array but do nothing.

Rendered UI: [L82-90](#) `highContrast` checkbox, [L92-100](#) `TTS` checkbox, [L102-106](#) `FontSizeSlider`, [L108-112](#) `BackButton`. No `TTSButton` rendered on this page.

BackButton ([L126](#) in `components.tsx`) calls `router.back()` (Next.js client-side back navigation).

[frontend/src/app/redirecting/page.tsx](#) — `"/redirecting"`

Purpose: Shown when a user tries to access an unknown URL. Counts down 3 seconds then redirects to `"/"`.

State: `seconds` — starts at `3`, counts down to `0`.

useEffect depends on `[seconds, router]`:

- If `seconds === 0` → calls `router.replace('/')` (replaces history entry so Back doesn't return here)
- Otherwise → sets a `setTimeout` of 1000ms to decrement `seconds` by 1
- Returns `clearTimeout` as cleanup to avoid stale timers

Layer 4 — Services

<frontend/src/app/services/jwtService.ts>

Handles all JWT storage and every authenticated API call. All functions here use the storage key `"pollinator_jwt_v1"`.

saveJwt(token: string): void (line 11)

Writes `token` to `localStorage` under key `"pollinator_jwt_v1"`. Wrapped in `try/catch` — silently ignores if `localStorage` is unavailable (e.g. private browsing).

getJwt(): string | null (line 17)

Reads from `localStorage.getItem("pollinator_jwt_v1")`. Returns `null` on failure or if not present.

clearJwt(): void (line 25)

Removes `"pollinator_jwt_v1"` from `localStorage`. Called automatically when a `401` response is received in `apiFetchAuthenticated`.

fetchJwtFromApi(numberToSend: number): Promise<string> (line 32)

Makes a raw (no auth) `POST` to `${API_BASE}/api` with body `{ sessionId: numberToSend }`.

1. Reads response as text
2. If `!res.ok` → throws with status and body text
3. Parses JSON as `ApiHandshakeResponse = { token: string }`
4. If parse fails → throws with parse error details
5. If `!data?.token` → throws `"JWT handshake returned no token"`
6. Calls `saveJwt(data.token)` — persists to `localStorage`
7. Returns the token string

ensureJwt(sessionId: number): Promise<string> (line 61)

The main entry point used by pages. Prevents duplicate simultaneous handshakes.

1. Calls `getJwt()` — if a token already exists in `localStorage`, **returns it immediately** (no network call)
2. If no token and no in-flight handshake (`jwtPromise === null`) → calls `fetchJwtFromApi(sessionId)`, stores the promise in the module-level `jwtPromise` variable
3. The `.finally()` clears `jwtPromise` back to `null` when the request settles, so future calls can make a new one

4. Returns `jwtPromise` — if a handshake was already in flight, all callers share the same promise

`apiFetchAuthed<T>(path: string, opts: RequestInit = {}): Promise<T>` (line 74)

Makes any authenticated API request. Used by `route/page.tsx` for `/api/start-game` and `/api/complete-route`.

1. Calls `getJwt()` — if null → throws "Missing JWT. Call `/api` first."
 2. Creates a `Headers` object from `opts.headers`
 3. Sets `Authorization: Bearer <jwt>`
 4. If `opts.body` is present and `Content-Type` is not already set → sets `Content-Type: application/json`
 5. Calls `fetch(`${API_BASE}${path}`, { ...opts, headers })`
 6. Reads response as text
 7. If `!res.ok` :
 - If status is `401` → calls `clearJwt()` first, then throws `Unauthorized: ...`
 - Otherwise → throws `API error: ...`
 8. Tries to `JSON.parse(text)` as type `T`
 9. If parse fails → throws with path and error details
 10. Returns the parsed data
-

[frontend/src/app/services/routeService.ts](#)

Manages the locally-stored list of completed routes and replay state. Uses the key `"completedRoutes_v1"` for both `localStorage` and cookies, and `"replay_v1"` for replay state.

Cookie helpers (private)

`getCookie(name)` : (renamed from `getCookieRoutesCompleted`) Reads a cookie by name using `document.cookie.match(new RegExp(...))`. Returns the decoded value or `null`. Returns `null` immediately if `typeof document === 'undefined'` (server-side render guard).

`setCookie(name, value, hours = 12)` : (renamed from `setCookieRoutesCompleted`) Sets a cookie with a 12-hour expiry by default. Calculates `expires` as `new Date(Date.now() + hours * 3600000).toUTCString()`. Returns immediately if server-side (`typeof document === 'undefined'`).

[L56-66](#) `getCompletedRoutes(): Route[]`

1. Tries `localStorage.getItem("completedRoutes_v1")` → parses JSON → returns `Route[]` array
2. If that fails (`localStorage` unavailable or parse error) → tries the cookie fallback via `getCookie`
3. If both fail → returns `[]`

Returns full `Route[]` objects (not `string[]`).

[L68-75](#) `setCompletedRoutes(completedRoutesIds: Route[]): void`

Takes `Route[]`. Writes the array to **both** `localStorage` and the cookie (dual-write for redundancy). Each in its own `try/catch`.

[L77-90](#) `addCompletedRoute(newRoute: Route): void`

1. [L78](#) Guards: if `!newRoute` → returns immediately
2. [L79](#) Reads current list via `getCompletedRoutes()`
3. [L82–84](#) Deduplicates by `routeId` — if not already in the list → pushes it and calls `setCompletedRoutes()`
4. [L86–88](#) Dispatches `CustomEvent("completedRoutesUpdated", { detail: completedRoutes })` on `window` — lets the `pollinator-collection` page update live without a page refresh

Note: In `route/page.tsx`, `addCompletedRoute` is called with the **full Route object**. Deduplication is by `routeId`.

[L34–41](#) `setReplayRoute(route: Route | null): void`

Saves the route as JSON to both `localStorage` and a cookie under key `"replay_v1"`. Pass `null` to clear the replay state.

[L43–53](#) `getReplayRoute(): Route | null`

Reads from `localStorage` first, then cookie fallback, under key `"replay_v1"`. Returns `null` if not set or if JSON parse fails.

[L93–97](#) `getRandomInt(min, max): number`

Returns a random integer in `[min, max)`. **Not currently used** in any page — available as a utility.

[L99–102](#) `getRandomRoute(availableRoutes: Route[]): Route`

Returns a random item from `availableRoutes` using `getRandomInt`. **Not currently used** in any page — backend handles route selection.

[L104–106](#) `filterAvailableRoutes(allRoutes, completedRouteIds): Route[]`

Filters a `Route[]` to exclude routes whose `routeId` is in `completedRouteIds`. **Not currently used** in any page — available as a utility.

Layer 5 — Shared UI Components

[frontend/src/app/components.tsx](#)

All reusable UI components live in [frontend/src/app/components.tsx](#).

Component	Line	What it renders
<code>LogoImage</code>	L8	<code></code> with class <code>logo_image</code>
<code>TextField({ labelText })</code>	L41	<code><h1 className="route_text">{labelText}</h1></code>
<code>RouteHeader({ headerText, dataTestId })</code>	L45	<code>{headerText}</code>
<code>EventButton({ buttonText, onButtonClick, isDisabled })</code>	L49	<code><button className="interactive_button"></code> — <code>isDisabled</code> prop disables the button

SurveyButton({ buttonText, onButtonClick, dataTestId })	L61	Same as EventButton but a separate component
TTSButton({ text })	L69	<button className="TTS-Button"> with TTS icon. Cancels in-progress speech then speaks text at rate = 0.75. See the TTS section for full detail.
LinkButton({ buttonText, linkTo, dataTestId })	L84	Next.js <Link href={linkTo}> — validates that linkTo is a real page
HomeButton({ onButtonClick, dataTestId })	L92	Next.js <Link href="/home"> with class interactive_button home_button
ReplayButton({ onButtonClick, dataTestId })	L100	<Link href="/route"> styled as button, renders "Replay" text
PollinatorImage({ coords, altText, isHidden })	L109	Sprite-sheet image — see detail below
BackButton({ onButtonClick })	L126	<button> that calls onButtonClick() then router.back()
SettingsButton({ dataTestId })	L141	Next.js <Link href="/accessibility"> showing a ⚙ gear icon
CollectorPopUp({ pollinatorName, imageCoords, onContinue, isNew })	L149	Full-screen overlay — shows pollinator name, image, "Continue" button. When isNew: true, adds "Adding {name} to your pollinator collection..."
IncrementButton({ onButtonClick })	L194	<button class="survey_button increment"> with plus icon
DecrementButton({ onButtonClick })	L202	<button class="survey_button decrement"> with minus icon
PartySizeSurveyPopUp({ onContinue })	L210	Full-screen overlay. "Optional Survey". Counters for children + adults, floored at 0, capped at 20 . Pre-fills from getSurveyResponse() on mount. Submit calls setSurveyResponse() then onContinue().
TextInput({ id, isDisabled, onValueChange, minLength, maxLength })	L20	Controlled numeric text input for session entry. inputMode="numeric", strips non-digit chars, has autoFocus.
FontSizeSlider({ value, onChange })	L331	Range input snapping to 75/100/125/150%. Sets --font-scale CSS custom property on <html> to scale all rem-based dimensions.

[L109–124](#) PollinatorImage({ coords, altText, isHidden }) : The sprite-sheet image component. Parses the coords string "Y.X" by splitting on "." :

- [L111](#) xCoord = parseInt(split[1]) * -100 (pixel offset for background-position X)
- [L112](#) yCoord = parseInt(split[0]) * -100 (pixel offset for background-position Y)
- [L113](#) Special case: if yCoord === 0 , forces xCoord = 0 as well (resets to origin for the starting node)

- [L114–123](#) Renders a `<div role="img">` with `backgroundImage: url(/images/spritesheet_transparent.png)` and `backgroundPosition: "${xCoord}px ${yCoord}px"`
- [L121](#) If `isHidden: true` → applies `filter: 'brightness(0) saturate(100%)'` (solid black silhouette)

[frontend/src/app/services/surveyStorageService.ts](#) (NEW)

Manages the locally-stored party size survey response. Uses `localStorage` key `"partySizeSurveyResponse"` .

`getSurveyResponse()`

Reads and parses JSON from `localStorage`. Validates that `children` , `adults` , and `totalPartySize` are all numbers. Returns `{ children, adults, totalPartySize }` or `null` if not set, parse fails, or validation fails.

`setSurveyResponse(children, adults, totalPartySize)`

Saves `{ children, adults, totalPartySize: children + adults, submittedAt: ISO string }` to `localStorage` under `"partySizeSurveyResponse"` .

`sendSurveyResponseToDatabase(token: string): Promise<void>`

Calls `getSurveyResponse()` . If a valid result exists, calls `apiFetchAuthenticated("/api/add-children-adults", { method: "POST", body: { numChildren, numAdults } })` . Ignores errors (fire-and-forget).

Full User Flow — Visual Summary

```

User opens app
|
├─ Any unknown URL → middleware.ts → redirect to /redirecting → countdown 3s →
  redirect to /
  |
  └─ "/" (page.tsx)
      |
      | User types session code (digits only, max 9, must be exactly 9)
      | Clicks "Submit"
      | POST /api { sessionId } → backend Validate handler
      | On success: localStorage["authToken"] = token, localStorage["sessionId"] =
code
      | Shows HomeButton
      |
      └─ "/home" (home/page.tsx)
          |
          | Reads getCompletedRoutes() from localStorage/cookie
          | POST /api/get-pollinator-names (authenticated) → count of pollinators
          | Shows "Pollinators Collected: X/<total from API>"
          |
          | └─ "Start New Pollinator Path" → "/route"
          |
          | └─ "Accessibility" → "/accessibility"
  
```

```

    | Settings saved to localStorage["accessibilitySettings"]
    | CSS classes toggled on <html> element live
    |
    └─ "Pollinator Collection" → "/pollinator-collection"
        Reads + sanitizes completedRoutes from localStorage/cookie
        Displays discovered (with image) + undiscovered (silhouette)
        Live-updates on "storage" and "completedRoutesUpdated" events

"/route" (route/page.tsx)
|
├─ On mount:
|   localStorage["authToken"] → saveJwt() → localStorage["pollinator_jwt_v1"]
|   | If "authToken" missing → show error, stop
|   |
|   apiFetchAuthed POST /api/start-game
|   | Sets Authorization: Bearer <token>
|   | On 401: clearJwt(), throws
|   |
|   setRoute([result.player.activeRoute])
|   setPollText(routeNodes[0].text)
|   setImageCoords(routeNodes[0].id)
|
├─ User navigates stops (routeIndex 0 → routeNodes.length-1):
|   Next/Previous buttons call handleClick(1) / handleClick(0)
|   routeIndex useEffect updates pollText + imageCoords each step
|
├─ At last routeNode (isCollectionTime):
|   CollectorPopUp appears → "Continue" dismisses it (collectorPopUpGuard=true)
|
├─ User navigates fact cards (routeIndex routeNodes.length → end):
|   RouteFactHTML rendered, no image, factHeader shown
|
├─ At last fact (isRouteEnd + isSurveyTime):
|   PartySizeSurveyPopUp appears → "Submit" dismisses (surveyPopUpGuard=true)
|   "Start New Route" button visible
|
└─ User clicks "Start New Route" → handleCompleteRoute()
    addCompletedRoute(lastRouteNode.text) → localStorage + cookie + CustomEvent
    POST /api/complete-route (authenticated)
    result.newActiveRoute → setRoute([newRoute]), reset routeIndex=0

```

Text-to-Speech (TTS)

TTS reads the current route or fact node text aloud when the user navigates — but **only if TTS is enabled** in accessibility settings. It is opt-in, gated by `localStorage["accessibilitySettings"].TTS`, which is set by `accessibility/page.tsx`.

Where TTS Lives

File	Lines	What it does
------	-------	--------------

route/page.tsx	L20 , L22-51 , L97-114 , L273 , L333 , L401 , L466	ttsEnabled state, three helper functions, cleanup useEffect, wiring into navigation, and conditional TTSButton render in all four layout functions
components.tsx	L69-82	TTSButton component — the manual replay button a user can tap to re-hear the current text
accessibility/page.tsx	L20 , L22-43 , L87-98	The TTS checkbox — saves the TTS boolean into localStorage["accessibilitySettings"]

Step 0 — Enabling TTS (Accessibility Page)

Before TTS does anything on the route page, the user has to turn it on. That happens in [accessibility/page.tsx](#).

[L20](#) — React state:

```
const [TTS, setTTS] = useState(false);
```

[L22-32](#) — Load from localStorage on mount:

```
useEffect(() => {
  const savedSettings = localStorage.getItem('accessibilitySettings');
  if (savedSettings) {
    const settings = JSON.parse(savedSettings);
    setTTS(settings.TTS ?? false);
  }
  setAccessibilitySettingsIsLoaded(true);
}, []);
```

When the page mounts, it reads any previously saved settings and hydrates state. The `accessibilitySettingsIsLoaded` flag prevents the next `useEffect` from overwriting storage before this load completes.

[L34-43](#) — Save to localStorage on change:

```
useEffect(() => {
  if (!accessibilitySettingsIsLoaded) return;
  localStorage.setItem('accessibilitySettings', JSON.stringify({ highContrast, TTS,
... }));
}, [highContrast, TTS, buttonPressReadAloud, autoReadAloud,
accessibilitySettingsIsLoaded]);
```

Any time a setting changes (including `TTS`), the whole settings object is serialized and written to `localStorage`. The route page reads this key to know whether to speak.

[L87-98](#) — The checkbox:

```
<input type="checkbox" checked={TTS} onChange={(e) => setTTS(e.target.checked)} />  
Text-to-Speech
```

Toggling the checkbox calls `setTTS`, which triggers the save `useEffect` above.

Step 1 — `ttsEnabled` State in `route/page.tsx`

[L20](#) — React state:

```
const [ttsEnabled, setTtsEnabled] = useState(false);
```

Holds whether TTS is on for this render. It starts `false` and is set on mount by reading `localStorage`. It's passed as a prop down into each layout function so they know whether to render the `TTSButton`.

Step 2 — Four Functions in `route/page.tsx`

1. [L22-25](#) — Unmount cleanup `useEffect`

```
useEffect(() => {  
  setTtsEnabled(isTTSEnabled());  
  return () => window.speechSynthesis.cancel();  
}, []);
```

Runs once on mount. First, it reads `localStorage` to set `ttsEnabled`. Then it returns a cleanup function — React calls this when the component unmounts (when the user navigates away from `/route`). The cleanup calls `window.speechSynthesis.cancel()`, which stops any in-progress or queued speech immediately. Without this, audio would keep playing after the user leaves the page.

2. [L27-33](#) — `isTTSEnabled()`

```
function isTTSEnabled(): boolean {  
  try {  
    const saved = localStorage.getItem('accessibilitySettings');  
    if (saved) return JSON.parse(saved).TTS ?? false;  
  } catch {}  
  return false;  
}
```

Reads `localStorage["accessibilitySettings"]` (written by `accessibility/page.tsx`) and extracts `.TTS`. The `?? false` nullish coalescing means it defaults off if the key is absent. The `try/catch` silently handles malformed JSON (e.g. storage cleared mid-session). Called both on mount (to initialize `ttsEnabled` state) and inside `speakIfEnabled()` (to re-check at speak time).

3. [L35-42](#) — `getTextForIndex(index)`

```
function getTextForIndex(index: number): string {  
  if (!route || route.length === 0) return "";
```



```

const totalRoute = route[0].routeNodes.length;
const factIndex = index - totalRoute;
if (index < totalRoute) return route[0].routeNodes[index].text;
if (route[0].factNodes && factIndex < route[0].factNodes.length) return
route[0].factNodes[factIndex].text;
return "";
}

```

`routeIndex` is a single counter that spans two arrays back-to-back: `routeNodes` (0 → N-1) then `factNodes` (N → N+M-1). This function converts a flat index into the correct text string from whichever array it falls in.

Example with a 3-stop route and 2 fact cards:

```

index 0 → routeNodes[0].text
index 1 → routeNodes[1].text
index 2 → routeNodes[2].text ← last route node
index 3 → factNodes[0].text
index 4 → factNodes[1].text ← route end

```

4. [L44-51](#) — `speakIfEnabled(text)`

```

function speakIfEnabled(text: string) {
  if (isTTSEnabled() && text) {
    window.speechSynthesis.cancel();
    const utterance = new SpeechSynthesisUtterance(text);
    utterance.rate = 0.75;
    window.speechSynthesis.speak(utterance);
  }
}

```

The single point where all automatic TTS speech is triggered. It re-checks `isTTSEnabled()` at call time (not just on mount) so it stays in sync if the user switches settings mid-session in another tab. It cancels any in-progress speech first to prevent overlap on fast taps, then speaks at 75% speed (`rate = 0.75`) for clarity. Uses the browser's built-in Web Speech API — no library or external dependency.

Step 3 — Wiring into [L97-114](#) `handleButtonClick`

Button	i value	TTS behavior
Previous	0	<code>newIndex = routeIndex - 1 → speakIfEnabled(getTextForIndex(newIndex)) → setRouteIndex(newIndex)</code>
Next	1	<code>newIndex = routeIndex + 1 → speakIfEnabled(getTextForIndex(newIndex)) → setRouteIndex(newIndex)</code>
Home	2	<code>window.speechSynthesis.cancel()</code> immediately → <code>handleHomeClicked()</code>

Why `newIndex` before speaking: TTS speaks the *destination* node's text. React state updates are async — `setRouteIndex(newIndex)` doesn't take effect until the next render. So `newIndex` is held in a local

variable and passed directly to `getTextForIndex` , before being handed off to React.

Why cancel inline on Home (L111): The Home button navigates away, which eventually unmounts the component and triggers the cleanup. But there's a React rendering delay. Calling `cancel()` inline stops speech the moment the user taps, not after the navigation completes.

Step 4 — TTSButton Component (L69–82 in `components.tsx`)

```
export function TTSButton({ text = "" }) {
  function handleClick() { // L70
    window.speechSynthesis.cancel();
    const utterance = new SpeechSynthesisUtterance(text);
    utterance.rate = 0.75;
    window.speechSynthesis.speak(utterance);
  }
  return (
    <button className='TTS-Button' onClick={handleClick}> // L78
      
    </button>
  );
}
```

This is the **manual replay button** — the user can tap it to re-hear the current stop's text at any time. It works identically to `speakIfEnabled` but without the enable-check (because it only renders when `ttsEnabled` is `true`). The `text` prop is always the current `pollText` value, updated each time `routeIndex` changes.

Render locations in `route/page.tsx` — all four layout functions:

Layout function	Line	Condition
RouteBeginningHTML	L273	{ttsEnabled && <TTSButton text={pollText} />}
RouteEndHTML	L333	{ttsEnabled && <TTSButton text={pollText} />}
RouteNormalHTML	L401	{ttsEnabled && <TTSButton text={pollText} />}
RouteFactHTML	L466	{ttsEnabled && <TTSButton text={pollText} />}

`ttsEnabled` is passed as a prop from `PageRoute` into each layout function. The prop is read from state ([L20](#)) which was set on mount by `isTTSEnabled()` . The `&&` short-circuit means no `TTSButton` is rendered at all when TTS is off — it doesn't exist in the DOM, it just doesn't appear.

Call Flow — Next Button Pressed

```
User taps "Next ►"
|
└─> handleClick(1)
    |
    ├── bounds check: routeIndex < routeNodes.length - 1 + factNodes.length
    └── const newIndex = routeIndex + 1
```

```

└─ speakIfEnabled(getTextForIndex(newIndex))
    └─ getTextForIndex(newIndex)
        └─ returns routeNodes[n].text or factNodes[n-total].text
    └─ if isTTSEnabled() && text:
        window.speechSynthesis.cancel()
        window.speechSynthesis.speak(utterance at rate 0.75)
└─ setRouteIndex(newIndex)
    └─ routeIndex useEffect fires → updates pollText + imageCoords → re-render
        └─ TTSButton now receives updated pollText for manual replay

```

What TTS Does NOT Do

- **No speech on initial load** — the first stop is displayed visually but not spoken on mount.
- **No speech on `handleCompleteRoute()`** — the "Loading new route..." transition text is not spoken.
- **Does not hook into the `routeIndex` `useEffect`** — speech is triggered only by user button presses, not by programmatic index changes.

localStorage & Cookie Keys Summary

Key	Where set	What it stores
"authToken"	page.tsx	JWT token (set by the session entry page only)
"sessionId"	page.tsx	The raw session code string the user typed
"pollinator_jwt_v1"	jwtService.ts	JWT token (used by all <code>apiFetchAuthenticated</code> calls)
"completedRoutes_v1"	routeService.ts (localStorage + cookie)	JSON array of completed Route objects
"replay_v1"	routeService.ts (localStorage + cookie)	JSON of the current Route to replay, or cleared when null
"partySizeSurveyResponse"	surveyStorageService.ts	JSON object { children, adults, totalPartySize, submittedAt }
"accessibilitySettings"	accessibility/page.tsx	JSON object { highContrast, TTS, buttonPressReadAloud, autoReadAloud }

Note: "authToken" (set by `page.tsx`) and "pollinator_jwt_v1" (read by `apiFetchAuthenticated`) are bridged on the route page — `route/page.tsx` reads "authToken" and copies it into "pollinator_jwt_v1" via `saveJwt()` on mount, so no second handshake is needed.

API Calls Made by the Frontend

Page / Service	Method	Endpoint	Auth	Purpose
page.tsx	POST	/api/	No	Validate session code, receive JWT stored under "authToken"
route/page.tsx (via apiFetchAuthenticated)	POST	/api/start-game	Bearer JWT	Initialize or rejoin player session, receive active route
route/page.tsx (via apiFetchAuthenticated)	POST	/api/complete-route	Bearer JWT	Mark route done, get next route
surveyStorageService.ts (via apiFetchAuthenticated)	POST	/api/add-children-adults	Bearer JWT	Send party size survey response to backend

Backend APIs Called by the Portal

The portal app actively calls several backend endpoints. See the Portal section below for full details.

Method	Endpoint	Auth	Purpose
POST	/api/get-children-adults	No	Look up children/adults data by session ID
POST	/api/get-child-adult-data-by-start-end-date	No	Get survey data within a date range

Backend APIs Not Called by the Frontend or Portal

Method	Endpoint	Auth	Purpose
GET	/api/health	No	Simple health check. Returns { status: "ok", timestamp }. Used to verify the server is running.
POST	/api/create-session	No	Creates or retrieves a session by the provided session ID. Returns { sessionId: number }.
POST	/api/get-pollinator-names	Bearer JWT	Returns all available pollinator names.
POST	/api/get-sessions-playerids-by-child-size	No	Sessions/players filtered by child count.
POST	/api/get-sessions-playerids-by-family-size	No	Sessions/players filtered by adult count.
POST	/api/get-sessions-playerids-by-adult-size	No	Sessions/players filtered by adult size.
POST	/api/get-child-adult-data-to-end-date	No	Survey data up to a given end date.

POST	/api/get-child-adult-data-to-forever	No	Survey data from a start date onwards.
------	--------------------------------------	----	--

Portal App

The portal is a **separate Next.js app** at `Pollinator-Habitat/portal/` . It is an admin-facing interface for Conner Prairie staff to query session data. It does not share routing, state, or services with the main player frontend.

Architecture

```
portal/src/app/
├── layout.tsx           - Root HTML shell
├── page.tsx             - "/" Portal main page
├── components.tsx       - Shared portal UI components
└── globals.css
portal/src/services/
├── csvExportService.ts   - Formats survey/route rows and triggers CSV download
├── inputValidator.ts     - Session ID sanitization + validation + date
validation
├── routeDataFetchService.ts - Parallel fetch wrappers for the 10 route-stat
endpoints
└── surveyDatafetchService.ts - API fetch functions for survey data
portal/testing/
├── surveyPortalStructure.test.tsx - Structural render tests
├── surveyPortalFunction.test.tsx - Functional input and search tests
├── exportTool.test.tsx         - CSV export service tests
├── inputValidator.test.tsx     - Input validation unit tests
├── layout.test.tsx            - Layout render tests
├── portalMenu.test.tsx        - Portal menu rendering tests
├── splashPage.test.tsx        - Portal splash page rendering tests
└── UAT/ExportSearchResults.test.tsx - UAT smoke test
```

[portal/src/app/page.tsx](#) — Portal Main Page

Purpose: Admin entry point. Lets staff search session data by session ID or date range.

State:

Variable	Type	Purpose
sessionId	string	Session ID search field
startDate	string	Start date (ISO YYYY-MM-DD), defaults to yesterday
endDate	string	End date (ISO YYYY-MM-DD), defaults to today
searchResults	SurveyDataRow[]	Table rows returned from search
loading	boolean	Disables search button during fetch

error	string null	Error message shown in UI
isRangeEnabled	boolean	Whether the end date is enabled (date range mode)

Search logic:

- If `sessionId` has any content → session ID search is primary; date search is ignored
- If `sessionId` is empty → date search is used
- `canSearch : hasSessionIdInput ? hasSessionIdCriteria : hasDateCriteria`
- `hasSessionIdCriteria : isValidSessionIdInput(sessionId)` — requires exactly 9 digits
- `hasDateCriteria : start date is valid; if isRangeEnabled , both dates must be valid`

`handleSearch()` :

- If date mode: calls `fetchSurveyDataByDateRange(startDate, end)` where `end` is `endDate` if `isRangeEnabled` , else `startDate` (single-day)
- If session ID mode: calls `fetchChildrenAdultsBySessionId(sessionId)`
- Updates `searchResults`

`handleSearchReset()` : Clears `sessionId`, `startDate`, `endDate`, `isRangeEnabled`, `error`.

`handleTableReset()` : Clears `searchResults` and `error`.

[portal/src/app/components.tsx](#) — Portal UI Components

Component	What it renders
LogoImage	<code></code> (changed from CPLOGO.png)
PortalText({ labelText })	<code><h1 className="portal_text">{labelText}</h1></code>
SearchHeader({ headerText })	<code><h2 className="search_header">{headerText}</h2></code> — NEW
PortalHeader({ headerText })	<code>{headerText}</code>
InteractiveButton({ buttonText, onClick, disabled })	<code><button className="interactive_button"></code>
ResetButton({ buttonText, onClick, disabled })	<code><button className="Reset-Button"></code> — NEW
SearchResultsTable({ searchResults })	Table with columns: Session ID, Date, Adults, Children, Total. Shows placeholder row "No results" when empty. — NEW
DateInput({ value, onChange, disabled, dataTestId })	<code><input type="date"></code> in a container div — NEW
SessionIdInput({ value, onChange, onSubmit, dataTestId })	Controlled <code><input type="text"></code> with <code>maxLength={9}</code> (was 10), passes through <code>sanitizeSessionIdInput()</code>

[portal/src/services/inputValidator.ts](#)

`sanitizeSessionIdInput(input: string): string`

Strips all non-digit characters (`replace(/\D/g, '')`) and slices to **9** characters (was 10). Used in `SessionIdInput` 's `onChange` handler.

`isValidSessionIdInput(sessionId: string): boolean`

Returns `true` only when `sessionId.length === 9` (was 10). Used by `PortalPage` to enable/disable the Search button.

`isValidDateInput(date: string): boolean (NEW)`

Validates format `^20\d{2}-\d{2}-\d{2}$` and checks that `new Date(date)` is a valid date. Used by the portal page to gate date-based searches.

[portal/src/services/surveyDatafetchService.ts](#) (NEW)

Exports type:

```
type SurveyDataRow = { sessionId: string; date: string; numAdults?: number | string; numChildren?: number | string }
```

`fetchSurveyDataByDateRange(startDate, endDate): Promise<SurveyDataRow[]>`

`POST /api/get-child-adult-data-by-start-end-date` . Validates JSON response. Maps `dataLoad` array to `SurveyDataRow[]` with `sessionId` , `date` (from `createdAt`), `numAdults` , `numChildren` .

`fetchChildrenAdultsBySessionId(sessionId): Promise<SurveyDataRow[]>`

`POST /api/get-children-adults` . Aggregates all player results into one row — totals adults and children across players. Returns a single row with the session's date, or empty array if no data.

Frontend Services Reference

Location: `frontend/src/app/services/`

`jwtService.ts`

Manages JWT lifecycle for the game frontend.

Export	Description
<code>saveJwt(token)</code>	Write JWT to <code>localStorage["pollinator_jwt_v1"]</code>
<code>getJwt()</code>	Read JWT from <code>localStorage</code>
<code>clearJwt()</code>	Remove JWT from <code>localStorage</code> (called on 401)
<code>fetchJwtFromApi(sessionId)</code>	<code>POST /api</code> → get token → save → return
<code>ensureJwt(sessionId)</code>	Return existing JWT or fetch a new one. Deduplicates concurrent calls via a shared promise.
<code>apiFetchAuthenticated<T>(path, opts)</code>	Authenticated fetch wrapper — injects <code>Authorization: Bearer</code> header, clears JWT on 401

Storage key: `pollinator_jwt_v1` in `localStorage`.

Concurrency: `ensureJwt` uses a module-level `jwtPromise` variable to prevent multiple simultaneous handshake calls if multiple components call it at the same time.

`routeService.ts`

Manages completed route state and replay selection using both `localStorage` and cookies (fallback).

Export	Description
<code>getCompletedRoutes()</code>	Read completed routes from <code>localStorage</code> (cookie fallback)
<code>setCompletedRoutes(routes)</code>	Write to <code>localStorage</code> AND cookie
<code>addCompletedRoute(route)</code>	Append to completed list (dedup by <code>routeId</code>), fires <code>completedRoutesUpdated</code> custom event
<code>setReplayRoute(route)</code>	Save a route to <code>localStorage</code> + cookie as the replay target
<code>getReplayRoute()</code>	Read the current replay route
<code>getRandomRoute(routes)</code>	Return a random item from the array
<code>filterAvailableRoutes(all, completedIds)</code>	Filter out already-completed routes

Storage keys:

- `completedRoutes_v1` — `localStorage` + cookie, 12-hour expiry
- `replay_v1` — `localStorage` + cookie

Cookie fallback: Routes are stored in both `localStorage` and cookies. The cookie acts as a fallback for browsers that restrict `localStorage` (e.g., private browsing modes).

`surveyStorageService.ts`

Manages the party size survey response.

Export	Description
<code>getSurveyResponse()</code>	Read { <code>children</code> , <code>adults</code> , <code>totalPartySize</code> } from <code>localStorage</code>
<code>setSurveyResponse(children, adults, total)</code>	Write survey response with timestamp to <code>localStorage</code>
<code>sendSurveyResponseToDatabase()</code>	POST <code>/api/add-children-adults</code> with stored response. Silently ignores errors.

Storage key: `partySizeSurveyResponse` in `localStorage`.

Shape stored:


```
{ "children": 2, "adults": 1, "totalPartySize": 3, "submittedAt": "2026-04-01T12:00:00.000Z" }
```

Components Reference

Location: `frontend/src/app/components.tsx`

19 exported components, all client-side (`"use client"`).

Component	Props	Description
LogoImage	—	Conner Prairie logo image
TextInput	<code>id?</code> , <code>isDisabled?</code> , <code>onValueChange</code> , <code>minLength?</code> , <code>maxLength?</code>	Numeric-only text input (strips non-digits)
TextField	<code>labelText?</code>	<code><h1></code> text display (route text rendering)
RouteHeader	<code>headerText?</code> , <code>dataTestId?</code>	<code></code> for route section headers
EventButton	<code>buttonText?</code> , <code>onButtonClick?</code> , <code>dataTestId?</code> , <code>isDisabled?</code>	Primary interactive button
SurveyButton	<code>buttonText?</code> , <code>onButtonClick?</code> , <code>dataTestId?</code>	Survey-specific button (same style as EventButton)
TTSButton	<code>text?</code>	Reads text aloud via Web Speech API at 0.75x speed
LinkButton	<code>buttonText?</code> , <code>linkTo?</code> , <code>dataTestId?</code>	Next.js <code><Link></code> styled as a button. Validates <code>linkTo</code> is a known route.
HomeButton	<code>onButtonClick?</code> , <code>dataTestId?</code>	Links to <code>/home</code> , styled left-aligned
ReplayButton	<code>onButtonClick?</code> , <code>dataTestId?</code>	Links to <code>/route</code> for replaying
PollinatorImage	<code>coords?</code> , <code>altText?</code> , <code>isHidden?</code>	Renders pollinator from spritesheet using CSS <code>backgroundPosition</code> . <code>isHidden</code> applies silhouette effect.
BackButton	<code>onButtonClick?</code>	Calls <code>router.back()</code> + optional callback
SettingsButton	<code>dataTestId?</code>	Gear icon link to <code>/accessibility</code>
CollectorPopUp	<code>pollinatorName?</code> , <code>imageCoords?</code> , <code>dataTestId?</code> , <code>onContinue?</code> , <code>isNew?</code>	Full-screen popup when a pollinator is discovered
IncrementButton	<code>onButtonClick?</code> , <code>dataTestId?</code>	+ button with plus-arrow SVG icon
DecrementButton	<code>onButtonClick?</code> , <code>dataTestId?</code>	– button with minus-arrow SVG icon

PartySizeSurveyPopUp	onContinue?, dataTestId?	Full party size survey with increment/decrement controls for children and adults. Internal state; saves to localStorage on submit.
FontSizeSlider	value: number, onChange: (v: number) => void	Range input (75/100/125/150%). Sets --font-scale CSS property on <html> to scale all rem dimensions.

PollinatorImage coord system: coords is a "row.column" string (e.g. "5.2" = Butterfly). The component converts this to CSS backgroundPosition: "-200px -500px" to show the correct 100×100px cell from the spritesheet. Special case: "0" always maps to position 0, 0 .

Note: The guard that previously threw when NEXT_PUBLIC_API_URL was empty has been removed. Both fetchSurveyDataByDateRange and fetchChildrenAdultsBySessionId now allow an empty API_BASE , producing relative URLs (e.g. /api/...) that work via the nginx reverse proxy in both dev and production.